Bridge, Not Edge: An Independent Re-measurement of KRAKEN v1.1 against SESTINA v1.0

Dylan Nguyen

2 September 2026

Abstract

The KRAKEN v1.1 paper reports that its engine beats SESTINA v1.0 by +2.3466 sets per game, taking 63.0% of decided sets and 80.4% of games over 10,000 games, measured in its own arbiter through a process bridge into SESTINA v1.0's engine. We re-measure the pairing in both engines. Inside FishLab's arbiter, with KRAKEN hosted exactly as its integration guide requires, KRAKEN loses under each of the four rule dialects tested: −0.648 [−0.740, −0.556] sets per game under FishLab's rules and −0.759 under its own declare-on-turn rule, over 17,400 games. Inside KV's own arbiter, with their bridge as published, the paper's figure reproduces on fresh deals (+2.446). The discrepancy is one defect in that bridge: it resets SESTINA v1.0 with the hand it holds *now* where the engine expects the hand it was *dealt*, which corrupts every legality certificate the belief rebuilds in a half-suit that seat has since gained or lost a card in. Passing the dealt hand and replaying the identical deals moves the result by −3.149 [−3.307, −2.991] sets per game; KRAKEN then loses in their arbiter too (−0.570 on 8,000 further games), SESTINA v1.0's declaration accuracy returns to its native level, and the two arbiters agree under the matched dialect. A decision-level probe shows the defect on single decisions. The paper's forced-endgame repair and its per-decision seed schedule are priced and are immaterial. KRAKEN also loses to every earlier release of FishLab's FishBot lineage from v0.4 upward. Nothing here bears on KRAKEN's internal results; the cross-engine claim, and what the paper builds on it, does not survive.

1 Introduction

Two engines for six-player Canadian Fish were written independently in 2026: FishLab, whose current agent is SESTINA v1.0 (developed as FishBot v0.7; [2]), and the Python engine of KV (Kausthubh Veldanda), whose current agent is KRAKEN (v1.0 in code, v1.1 in the paper; [1]; the deployed configuration is the same). In August the two projects built bridges so that each bot could play inside the other's arbiter, and the KRAKEN paper reports the result of its exhibition as its cross-engine headline: over 10,000 games under its rules, KRAKEN beat SESTINA v1.0 by +2.3466 sets per game, won 80.4% of games, and declared half-suits correctly 96.5% of the time against SESTINA v1.0's 78.9%. The paper builds a mechanism story on those rows — 57% of the margin is attributed to "knowing when a half-suit is finished" — and its package README extends the claim to FishLab's own dialect, giving +2.375 sets per game when both sides may declare off-turn.

The size of the claim is what makes it worth checking. SESTINA v1.0's own evaluation resolves edges of a few percentage points against its predecessors; an opponent taking four games in five

would be a different order of strength from anything either lineage has produced. The paper is careful about what the number is: cross-engine strength against one opponent, not a global ranking. But a cross-engine number is a statement about the bridge as well as the engines, as the paper itself says, and the bridge is the one component neither project had tested from the other side.

This report does that. It uses four instruments, in the order a sceptic would reach for them:

1. the same match inside FishLab's arbiter, with KRAKEN played through FishLab's bridge to it (Section 4.1);
2. a replication inside KV's own arbiter, with their engine, their rules and their bridge, on fresh deals (Section 4.2);
3. one-line variants of their bridge, each changing exactly one thing, replayed on identical deals so the difference is paired (Section 4.3);
4. a probe that feeds every native SESTINA v1.0 decision through KV's own shim and compares the two answers decision by decision (Section 4.4).

Section 5 locates the defect in the two code bases and explains why every check the paper ran passed anyway. Section 6 states what is refuted, what stands, what was not tested, and what a cross-engine measurement should require of its bridge. Every measurement in this document is a macro generated from an artifact in `research/kraken-check/` by `make_numbers.py`; the few figures quoted from the KRAKEN paper are transcribed in the same file under a comment naming their source.

Table 1: The claim against the measurements. Left: the paper's headline, from its own results file. Middle: the same pairing in FishLab's arbiter under FishLab's rules. Right: the same pairing in KV's arbiter after the one-line repair of Section 5, on fresh deals.

quantity	the paper (KV's arbiter, their bridge)	FishLab's arbiter, FishLab rules	KV's arbiter, bridge repaired
KRAKEN margin, sets per game	+2.3466 [+2.2928, +2.4004]	−0.648 [−0.740, −0.556]	−0.570 [−0.632, −0.508]
KRAKEN share of decided sets	63.0%	46.4%	46.8%
KRAKEN games won	80.4%	40.3%	41.7%
SESTINA declaration accuracy	78.9%	98.0%	96.5%
KRAKEN declaration accuracy	96.5%	98.6%	97.4%
SESTINA ask hit rate	48.1%	56.0%	54.7%
KRAKEN ask hit rate	51.7%	53.8%	52.5%
games	10,000	5,400	8,000

2 The engines, the rules, and the bridges

2.1 Two arbiters, two dialects

The game, in one paragraph. Fifty-four cards form nine *half-suits* of six; six seats in two teams of alternating seats are dealt nine cards each. On turn a player *asks* an opponent for a named card: it is legal only if the asker holds another card of that half-suit and not the card itself; a hit transfers

the card and keeps the turn, a miss passes the turn to the opponent asked. A team scores a half-suit by *declaring* it: naming which of its own three seats holds each of the six cards. Every action is public, so the whole hidden state is the deal, and an engine's belief is a posterior over deals under the constraints the public record implies: a hit pins a card, a miss excludes a seat, and every ask certifies that the asker held another card of that half-suit at that moment (a *legality certificate* in what follows). The *arbiter* is the program that runs the game and scores it; a *bridge* is the code that lets one project's bot play inside the other's arbiter; a *dialect* is one project's set of choices on the rules the game leaves open.

Both engines implement this game with the no-bluff ask rule and the award rule for misdeclarations: a declaration that is wrong in any way — a card with an opponent (an *ownership* error) or all six within the team but the split misstated (an *allocation* error) — scores the half-suit for the other team. (KV's engine scored a right-team, wrong-split declaration as a void until its v0.6; the paper re-measured everything under the award rule, and every result here uses it.) They differ in four places that matter to what follows.

Out-of-turn declaration. FishLab's default rule lets any seat declare at any moment, and the engine polls every seat before every move. KV's baseline lets a seat declare only on its own turn (`claims_any_time = False`). FishLab exposes the latter as `--no-out-of-turn`.

Cardless seats. FishLab lets a cardless seat declare; KV's arbiter makes it pass. FishLab exposes the latter as `--no-cardless-declare`.

The forced endgame. When one team is out of cards, FishLab runs a willingness ladder over the declaring team: for each half-suit and each seat, at descending confidence thresholds, a seat that is willing declares, and only a last resort demands a best guess. Only the willingness bit crosses between teammates. In KV's arbiter the mover must claim one half-suit per turn, and the bridge chooses which.

What a wrong declaration reveals. KV's engine reveals the true holders at resolution; FishLab reveals nothing.

2.2 KRAKEN inside FishLab

FishLab's bridge, `engine/src/kv6.hpp`, launches KRAKEN's own package (`kraken.decide`, unmodified, under the vendored interpreter) as a line-oriented JSON service, one process per worker thread. It follows the three requirements KRAKEN's integration guide says a host must get right: every seat is polled off-turn through KRAKEN's proof-only channel; cards travel as names, with the half-suit permutation between the two decks derived at start-up from KRAKEN's own `{"op":"cards"}` table; and any error reply, illegal move or dead process aborts the run rather than substituting a move. KRAKEN has no clock, so no timeout is set. Because KRAKEN's belief cannot represent "resolved, holders unrevealed", the engine additionally hands it the true holders after a wrong declaration, out of band and to no policy of FishLab's — the one thing FishLab grew for the integration, and a gift to KRAKEN relative to FishLab's own rule. A full request–reply transcript of one deal is in the artifacts: 645 requests (167 on-turn decisions, 477 off-turn polls), 0 error replies. KRAKEN's own consistency checker, which rejects an event stream no deal could have produced, never fired in any run reported here.

2.3 SESTINA inside KV's arbiter

KV's bridge is `fish4/dylan_v07.py` driving `external_v07/shim_decide.cpp`, a one-shot binary compiled from FishLab's headers. It is stateless by construction: for every decision, a fresh process is started and fed

```
SPEC <the frozen v07 allparams spec>
RULES 9 0 1                 # 9 half-suits, on-turn declaration only, cardless OK
SEAT s
HAND <bitmask>              # the seat's hand NOW (Observation.hand)
SEED <one constant per game>
EV ASK|DECL|PASS ...        # every public event, running counts and score
TURN s
DECIDE TURN                 # or DECIDE FORCED <half-suit> / PASSTO
```

and the shim constructs the agent, calls `Agent::reset(seat, hand, rules, seed)`, replays the events through `observe()`, and answers one move. SESTINA v1.0 is never polled off-turn (their rules have no such channel); when the mover must declare, the bridge names the first live half-suit the mover holds a card in and asks for the agent's own proposal on it, falling back to a best guess (this is the paper's bridge revision 2; revision 1 passed the first live half-suit outright, and the paper prices the repair at -0.0784 sets per game). Every proposed move is legality-checked and a failure would be counted; the paper reports zero, and so does every run here.

2.4 Versions

KRAKEN is the package vendored at `engine/external/kraken` (v1.0; `V06_DEPLOYED` plus `claim_forced_exhaustive=1`). The upstream branch head `5da2040` (v1.1) was fetched and played on the same seed, same rotations and same opponent as the first FishLab block: the results are identical to the digit (38.8% against 38.8% of games, 4.145/4.855 against 4.145/4.855 sets, 26,256 against 26,256 asks, 2,423 against 2,423 declarations). The v1.1 paper says its deployed configuration is unchanged from v1.0; this confirms it. KV's package was itself called FishBot v0.6 before it became KRAKEN; the copy vendored at `engine/external/kv-fishbot-v06` is an earlier snapshot of that package whose policy code differs from KRAKEN's in more than the one flag, it is unrelated to FishLab's own FishBot v0.6, and it is written "KV FishBot v0.6" throughout. SESTINA v1.0 is the frozen specification in `engine/fishbot_v07.json`; the `allparams` form KV's bridge sends equals their `v07_spec.txt`. The paper pins FishLab at commit `d017fbc`; between that commit and the build used here the only changes under `engine/src/` to any file the policy depends on are two additive hooks (the true-holder callback above, and factory branches for external bots).

3 Methods

3.1 The FishLab battery

Each cell is a set of independent blocks of 100 deals played in all six seat rotations (600 games), so that each policy holds every hand-triple of a deal exactly three times and the deal's luck cancels. Each block has its own seed and is one invocation of `fish match` with `--json`; the row it returns, with its exact `argv`, is one line of `e1_results.jsonl`. A cell's estimate is the mean over its blocks and its interval is a Student t interval on the block means, which is deal-clustered by construction. Cells: KRAKEN against SESTINA v1.0 under FishLab's default rules, under

`--no-out-of-turn` (KRAKEN's dialect), under both that and `--no-cardless-declare`, and under `--legacy`; KRAKEN against KRAKEN as a symmetry control; KV FishBot v0.6 against SESTINA v1.0, once on its own seeds and once on the seeds of the default KRAKEN cell so that the two packages are compared on identical deals; and KRAKEN against FishBot v0.6, F-cheap (v0.6 with search), v0.5 and v0.4 to place it on this repository's ladder. 55 blocks, 33,000 games. Throughput was about 9 games per second on 9 threads while the other batteries ran alongside.

3.2 Replication in KV's arbiter

The driver reproduces the design of their `scripts4/mega_match.py` (fetched from upstream and kept under `upstream/scripts4/`): KRAKEN's `V06_DEPLOYED` against three copies of `dylan_v07`, the award rule, each deal played from both seat parities, their 600-ply cap, one JSON row per game. It runs their engine and their bridge unmodified, from the vendored package, with the shim compiled from this repository's headers; only the seeds are fresh. Each row additionally records, per side, asks and hits, declarations and their correctness, whether each declaration was forced (the mover had no legal ask), and the class of each wrong declaration: *ownership* (an opponent held a card of the half-suit) or *allocation* (the declaring team held all six and named the wrong split). Bridge fallbacks and unfinished games are counted; both are zero in every row. Intervals in KV's arbiter are normal, ± 1.96 standard errors on per-game margins; the two parities of a deal are nearly uncorrelated (worst $|\rho| = 0.044$ over the six runs) and deal-clustered half-widths agree with them to within 0.003 sets.

3.3 Bridge variants

Four variants of their bridge were run on the same deal seeds and agent seeds as the baseline, so that per-game differences are paired (three change one thing; the fourth combines two repairs), and a fifth run repeats the repaired bridge on fresh deals. Each subclasses their `DylanV07`:

dealt hand. `HAND` is the hand as dealt, recovered by walking the seat's public transfers newest-first from the current hand (a card taken by a successful ask is removed, one lost is restored, one retired by a declaration is restored to the seat that held it). Every decision also replays the result forward and checks that it reproduces the current hand; 0 mismatches occurred in 2,000 games.

ladder. When the mover must declare, it runs FishLab's forced ladder over every live half-suit (voluntary proposal, then willingness at the engine's descending thresholds, then the most confident best guess), through a shim extended with one command. 1,040 forced decisions went through it.

seed. The per-decision agent seed varies with the history length, so the fresh process does not restart the same random stream at every decision.

dealt hand and seed. Both repairs.

dealt hand, fresh deals. The dealt-hand bridge on 8,000 new games, unpaired.

3.4 The decision probe

`agree_probe.cpp` wraps native SESTINA v1.0 in FishLab's arbiter under KV's rules (declare on turn only) against KRAKEN. At each of its own on-turn decisions the wrapper also serialises the position exactly as `dylan_v07.py` does and asks KV's shim binary twice — once with the current hand, once with the hand as dealt — sending the native agent's exact seed (their bridge masks its seed to 63 bits; the probe does not, because the seed is not what is under test). It records whether the shim declares when the native agent does, whether the declarations match, whether

the asks match, and, using the true hands it has access to as the arbiter, whether each hypothetical declaration would have been right. The first decision of a game is a special case: the fresh shim process and the native agent then have identical random state, so if the serialisation is faithful they must answer identically. The probe was run twice, once with the native agents reused across deals as the production harness reuses them and once with them rebuilt for every game, because SESTINA v1.0's own report documents a rare cross-deal residual in reused agents (its §12.6).

4 Results

4.1 FishLab's arbiter

Table 2: Every cell of the FishLab battery. Blocks of 100 deals × 6 rotations; intervals are Student t on block means. "F-cheap" is FishBot v0.6 with SESTINA v1.0's search switched on.

A	B	rules	games	A wins	sets A−B per game	decl. acc. A / B (%)
KRAKEN	SESTINA	FishLab default	5,400	40.3%	−0.648 [−0.740, −0.556]	98.6 / 98.0
KRAKEN	SESTINA	declare on turn only	6,000	38.7%	−0.759 [−0.842, −0.676]	98.1 / 98.8
KRAKEN	SESTINA	on turn only, no cardless declaration	3,000	41.2%	−0.632 [−0.700, −0.564]	98.0 / 98.8
KRAKEN	SESTINA	legacy dialect	3,000	38.9%	−0.739 [−0.889, −0.590]	97.9 / 98.8
KRAKEN	KRAKEN	FishLab default (control)	1,200	50.0%	+0.000 [+0.000, +0.000]	98.7 / 98.7
KRAKEN	FishBot v0.6	FishLab default	3,000	44.1%	−0.440 [−0.650, −0.230]	98.4 / 98.5
KRAKEN	F-cheap	FishLab default	1,800	44.1%	−0.506 [−0.795, −0.216]	98.0 / 98.4
KRAKEN	FishBot v0.5	FishLab default	1,800	43.4%	−0.416 [−0.675, −0.156]	98.6 / 98.2
KRAKEN	FishBot v0.4	FishLab default	1,800	43.9%	−0.407 [−0.613, −0.201]	98.0 / 98.4
KV FishBot v0.6	SESTINA	FishLab default	3,000	42.3%	−0.495 [−0.608, −0.382]	98.5 / 97.9
KV FishBot v0.6	SESTINA	FishLab default, the KRAKEN cell's seeds	3,000	39.6%	−0.649 [−0.784, −0.514]	98.4 / 98.0

Table 2 is the whole battery. Against SESTINA v1.0, KRAKEN loses under every dialect: its share of games won runs from 38.7% to 41.2% and its margin from −0.759 to −0.632 sets per game over 17,400 games. The default-rules cell sits 75 of its own standard errors from the paper's figure, and the like-for-like cell, declaration on turn only, 84. KRAKEN's dialect makes its result slightly worse, not better; removing cardless declaration as well moves it by +0.127 in KRAKEN's favour, on different deals, which is within the between-seed variation the paired cell below exposes and in any case leaves the sign and the conclusion untouched; the legacy dialect sits with the on-turn-only

cells. The README's dialect table, which puts KRAKEN at +2.375 under FishLab's rules and +2.675 when nobody declares off-turn, is not approached under either.

The mirror cell is the control that the harness is symmetric: KRAKEN against itself is 50.0% to the digit (a tautology for a deterministic bot under a duplicate design, but it rules out a seat-order or orientation bug). The remaining cells place KRAKEN on this repository's ladder. It loses to FishBot v0.6 (−0.440), to F-cheap (−0.506), to v0.5 (−0.416) and to v0.4 (−0.407). The earlier KV FishBot v0.6 package, played through the same bridge, loses to SESTINA v1.0 by −0.495 on its own seeds but by −0.649 on the KRAKEN cell's seeds, and on those identical blocks its margin differs from KRAKEN's by −0.015 [−0.026, −0.005] sets per game: the two packages are indistinguishable once the deals are held fixed. The unpaired gap between the two cells was deal variation between seed ranges, and it is worth stating as a calibration of this battery: differences between two *unpaired* cells of a few blocks each should not be read below about 0.15 sets per game, and the paired instrument is the one to use for such contrasts. KRAKEN's declaration accuracy inside FishLab (98.6%) is higher than in its own arbiter (96.5% in the paper's journal); there is no sign that FishLab's hosting degrades it, although no probe of the kind Section 4.4 applies to SESTINA v1.0 was built for KRAKEN.

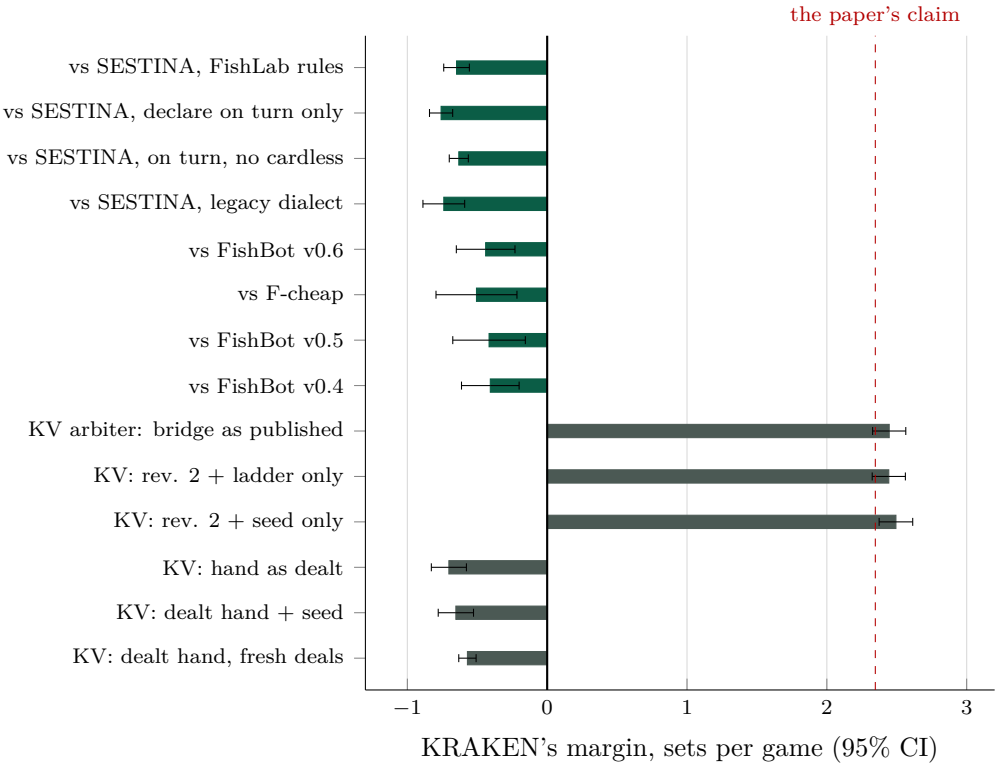


Figure 1: KRAKEN's margin in every non-control cell of both arbiters, sets per game with 95% intervals. Green: FishLab's arbiter. Grey: KV's arbiter, from the bridge as published (top three) to the repaired bridge (bottom three). The dashed line is the paper's headline. The KV FishBot v0.6 cells are in Table 2 only, since their bars would be that bot's margin, not KRAKEN's.

4.2 KV's arbiter reproduces the paper

With their engine, their rules and their bridge as published, on fresh deals, KRAKEN's margin is +2.446 [+2.327, +2.565] over 2,000 games, with 63.6% of sets and 81.7% of games (first row of

Table 3). The paper's journal gives +2.3466, 63.0% and 80.4%. SESTINA v1.0 declares correctly 78.0% of the time here against 78.9% there, and hits 48.0% of its asks against 48.1%. The paper's measurement is therefore reproducible on its own apparatus, and the disagreement with Section 4.1 is not noise on either side.

The instrumented rows say where the wrong declarations of SESTINA v1.0 come from. SESTINA v1.0 is forced to declare 0.52 times a game and gets 79.4% of those right; its *voluntary* declarations, the ones its own gate chose to make, are right only 77.8% of the time. Of its wrong declarations, 0.592 per game are ownership errors — half-suits an opponent still held a card of — against 0.285 allocation errors. Natively, the gate of SESTINA v1.0 essentially never declares a half-suit an opponent holds a card of; something is wrong with what it is being told about the table.

4.3 One line of the bridge

Table 3: Bridge variants inside KV's arbiter. All rows but the last are the same 2,000 games (1,000 deals × 2 parities, same deal and agent seeds), so the paired column is a per-game difference on identical deals. "K" is KRAKEN, "S" is SESTINA v1.0.

bridge	games	K margin	K wins	S decl. acc.	S ask hit	S own. err. per game	paired Δ vs rev. 2
as published (rev. 2)	2,000	+2.446 [+2.327, +2.565]	81.7%	78.0%	48.0%	0.592	—
hand as dealt instead of current hand	2,000	−0.703 [−0.828, −0.578]	39.5%	96.9%	54.9%	0.042	−3.149 [−3.307, −2.991]
rev. 2 + forced-endgame ladder only	2,000	+2.443 [+2.324, +2.562]	81.5%	78.1%	48.0%	0.592	−0.003 [−0.008, +0.002]
rev. 2 + per-decision seed only	2,000	+2.494 [+2.374, +2.614]	82.1%	78.2%	47.8%	0.579	+0.048 [−0.066, +0.162]
dealt hand and per-decision seed	2,000	−0.653 [−0.779, −0.527]	40.1%	96.6%	55.0%	0.044	−3.099 [−3.260, −2.938]
dealt hand, fresh deals (unpaired)	8,000	−0.570 [−0.632, −0.508]	41.7%	96.5%	54.7%	0.047	—

Table 3 prices each hypothesis. The forced-endgame ladder — the mechanism the paper's own bridge repair addressed — is worth −0.003 [−0.008, +0.002]: nothing. A varying seed is worth +0.048 [−0.066, +0.162]: nothing. Passing the hand as dealt is worth −3.149 [−3.307, −2.991] sets per game, and it reverses the sign of the result: KRAKEN's margin becomes −0.703 [−0.828, −0.578], its share of games 39.5%. SESTINA v1.0's declaration accuracy rises from 78.0% to 96.9% (voluntary 97.5%), its ownership errors fall from 0.592 to 0.042 per game, and its ask hit rate rises from 48.0% to 54.9% — the same corruption was costing it asks as well as declarations. Adding the varying seed on top changes little (−0.653). On 8,000 further fresh games the repaired bridge gives −0.570 [−0.632, −0.508].

Two things follow. First, KV's arbiter and FishLab's now agree. Under the FishLab dialect closest to KV's — declaration on turn only, no cardless declaration — FishLab gives

−0.632 [−0.700, −0.564], against −0.703 and −0.570 from the repaired bridge in KV's arbiter; the remaining differences of hosting are discussed in Section 6.2. Second, the paper's mechanism story — that SESTINA v1.0 finds cards at a comparable rate but does not know when a half-suit is finished — is describing a policy whose posterior had been corrupted before it was asked anything. Its 78.9% is the bridge, not the engine.

4.4 The same defect, decision by decision

Table 4: The decision probe: 120 games, 6,403 on-turn decisions of native SESTINA v1.0 in FishLab's arbiter under KV's rules, each also answered by KV's shim from the current hand and from the hand as dealt. Hypothetical declarations are scored against the true hands.

	native SESTINA	shim, current hand (rev. 2)	shim, hand as dealt
declarations over these decisions	580	793	673
of which wrong	8 (1.4%)	176 (22.2%)	38 (5.6%)
agrees with native on declare / not	—	6,127 (95.7%)	6,310 (98.5%)
declares when native does not	—	239	93
of which wrong	—	158	30
naming a half-suit an opponent holds	—	134	22
declines when native declares	—	26	0
both declare, different allocation	—	11	0
same ask as native (of 5,733)	—	4,141 (72.2%)	4,815 (84.0%)
first decision of a game identical, agents rebuilt per game (of 360)	—	360	360
agents reused across deals (of 360)	—	359	359

Table 4 removes the arbiter from the question altogether. Given the current hand, KV's shim declares 793 times where the native agent declares 580; 22.2% of the shim's declarations are wrong against 1.4% of the native agent's, and of the 239 declarations the shim makes when the native agent does not, 158 are wrong and 134 name a half-suit an opponent is holding. Given the hand as dealt, the shim never declines a declaration the native agent makes and never allocates one differently; its remaining excess declarations (93, 30 wrong) and the ask disagreements are consistent with the restarted random stream, the one piece of state a fresh process cannot reconstruct, and the seed schedule itself is priced at +0.048 in play. The first-decision row is the check on everything else in their serialisation: with the native agents rebuilt for every game, both versions of the shim answer what the native agent answers at all 360 first decisions. With the agents reused across deals, 359 of 360 agree; the one exception is a first decision at which the reused native agent itself differs from a freshly built one — the cross-deal residual of SESTINA v1.0's report — and at which the shim matches the fresh agent, so it is a property of FishLab's harness, not of the bridge.

5 The defect

5.1 What the bridge sends

Their `Observation.from_state` sets `hand = state.hands[player]`, the hand at this moment, and `DylanV07._feed` writes it to the `HAND` line. Their shim then boots the agent with

`agent->reset(seat, hand, rules, seed)` and replays the history. In FishLab, `reset` is called once, by `Game::setup`, with the hand as dealt, and the agent thereafter learns every movement from the events it observes. The two conventions coincide only until the seat's first transfer.

5.2 What the engine does with it

`Knowledge::init` (`engine/src/belief.hpp`) marks every card in the reset hand as owned by this seat and every other card as unknown with a candidate set that excludes this seat. Under the bridge's convention a card this seat has since taken from an opponent is therefore "mine since the deal", and a card it was dealt and has since lost is "never mine".

The damage is done when the history is replayed. `Knowledge::onEvent` (`belief.hpp:189-201`) turns every ask by a seat p in half-suit S into a legality certificate — p held some other card of S at that moment — built over the cards of S whose owner is still unknown and whose candidate set includes p (lines 189–201, verbatim):

```
// (C5): the asker held another card of S.  Vacuous if we already know one.
bool vacuous = false;
uint64_t D = 0;
for (int i = 0; i < SETSZ; i++) {
  int d = cardOf(S, i);
  if (d == e.card) continue;
  if (owner[d] == e.actor) { vacuous = true; break; }
  if (owner[d] == UNKNOWN && (mask[d] & (1u << e.actor))) D |= bit(d);
}
exclude(e.card, e.actor);                          // (C3) asker lacks the card
if (!vacuous && D) {
  if (__builtin_popcountll(D) == 1) setOwner(__builtin_ctzll(D), e.actor);
  else if (!dedupDisj) disj.push_back(Disjunction{uint8_t(e.actor), D});
```

Suppose KRAKEN's seat p asks for a card of S at ply 10, holding only the 3 of that suit, and at ply 40 SESTINA v1.0's seat takes that 3 from p. At the reconstruction for any decision after ply 40 the 3 is already `owner == me`, so when ply 10 is replayed it is excluded from D. The certificate is now a statement about the wrong cards; if only one card of S remains unknown with p in its candidate set, line 200 pins that card to p as a deduction — a card p never held. The exact counting and the Sinkhorn fit that SESTINA v1.0's declaration gate reads then reconcile a constraint set no deal satisfies, and the reconciliation shows up as ownership errors: half-suits declared as wholly held while an opponent still holds a card of them.

5.3 Why every check passed

The paper's checks could not see this, for three reasons. *Legality*: every move the shim returns is legality-checked and zero fallbacks are reported, but a policy with a corrupted belief still makes legal moves. *Decision-level verification was done for the other bridge*: the fresh-agent comparison of the paper's Appendix C (329 decisions, no mismatches) verifies KRAKEN's own transport, `fish4/decide.py`, against a fresh KRAKEN agent; for the v0.7 shim the paper and its bridge README report legality checks, the fallback counter, twenty mixed games with zero fallbacks, and a unit test of the forced-half-suit rule (`tests4/test_dylan_bridge.py`, fetched upstream), none of which compares a decision with what the native agent would have done. A test in the shape of Appendix C would not have caught it either unless the fresh agent were the *native* one: a fresh agent booted from the same current hand reproduces the defect and compares it with itself. *Forensics*: the paper examined SESTINA v1.0's errors closely (§11.9) and found that every one

of the 398 cards inside its ownership errors had never publicly moved; that ownership errors were 67 times as frequent, per ply of exposure, in half-suits KRAKEN had asked in as in half-suits it had not; and that of the declarations which followed both an ask by KRAKEN and a later capture of a card of that half-suit by SESTINA v1.0, 19.25% were ownership-wrong, against 3.16% for a capture alone. That is exactly the population this defect produces: KRAKEN's ask creates the certificate, SESTINA v1.0's later capture in that half-suit is what the current-hand reset mis-files, and the phantom deduction lands on a card that never moved. The paper read the signature as a flaw in SESTINA v1.0's inference ("as though 'at least one' had been 'exactly one' "); it is not what the code does.

5.4 What inherits it

Every cross-engine figure in the paper was taken through this bridge: the head-to-head, the declaration decomposition (57% of the margin as "declaration accounting"), the acquisition and turn-count tables, the README's dialect gaps, the pre-registered foreign-opponent checks R2 and R6, and the contestation-term port of Table 18, which measured a FishLab coordinate against a corrupted FishLab. Their website's exhibition reconstructs every session from the public log through the same `_feed`. The paper's bridge ledger defends the paired contrasts among these — the dialect gaps, R2, R6 — as bridge-independent because every arm shared the bridge. That holds only when the bridge's effect is additive, and a corruption of the opponent's posterior worth three sets a game is not: the README's own table has off-turn declaration costing KRAKEN -0.300 sets against the corrupted SESTINA v1.0, while against the native one in FishLab it is worth $+0.111$ to KRAKEN (Table 2). A gap measured against a corrupted opponent is not the gap against the real one. The bridge-revision-2 repair the paper describes is real but immaterial by comparison: it addressed which half-suit a forced best guess names, worth -0.0784 by the paper's own pairing and -0.003 for the whole ladder here.

6 Discussion

6.1 What is refuted, what stands, what was not tested

Refuted: that KRAKEN beats SESTINA v1.0 by about $+2.3466$ sets per game, takes 63.0% of sets, wins 80.4% of games, or has an eighteen-point declaration-accuracy edge over it. In both arbiters, once the bridge passes SESTINA v1.0 the hand it was dealt, KRAKEN loses by roughly -0.570 to -0.759 sets per game under each of the dialects tested (four in FishLab, KV's baseline in KV's arbiter) and wins about two games in five. The mechanism story built on the 78.9% row is refuted with it.

Standing: everything in the paper that does not pass through the bridge. Its internal lineage result ($+2.412$ sets per pair against its own v0.3 champion), its exact counting and sampling, its posterior-quality measurements, its endgame solver and its pre-registration discipline are untouched by anything here, and this report makes no claim about them.

Not claimed: a global ranking. One foreign opponent is one opponent, and SESTINA v1.0's own report declines the same claim for the same reason. KRAKEN's placement below FishBot v0.4 through v0.6 on FishLab's ladder is context, not a leaderboard.

6.2 Which arbiter is faithful

Neither arbiter is the game; each hosts one bot natively and the other through a bridge, and the question is what each bridge costs. KRAKEN is stateless by design — its guide says a host may send one line and close the pipe — so a bridge costs it nothing, and in FishLab it is additionally handed the true holders on a wrong declaration and declares more accurately than in its own arbiter. SESTINA v1.0 is stateful and searches; a fresh process per decision restarts its random stream, their forced endgame lets only the mover declare and from a best guess, and it is never polled off-turn. Even with the dealt hand, the probe shows the shim still declaring wrongly 5.6% of the time against 1.4% natively. SESTINA v1.0's ask hit rate in the repaired KV arbiter (54.9%) is not below FishLab's under the matched dialect (53.9%), so no residual shows there. Of the hosting differences, the two that could be varied one at a time are priced in Table 3 and are small; the rest are bounded only by the end-to-end comparison of matched-dialect cells in Section 4.3. The honest summary is that the corrected KV arbiter and FishLab give the same sign and nearly the same size, and that what little residual there is runs against SESTINA v1.0 rather than for it.

6.3 Fairness to KRAKEN in FishLab

The symmetric question is whether `kv6.hpp` does to KRAKEN what `dylan_v07.py` did to SESTINA v1.0. The evidence that it does not: KRAKEN's three stated requirements are met; its own event-stream checker never rejected a position; the transcript of a full deal is well-formed; it is handed information FishLab's own agents do not get; it has no clock; the harness is symmetric; and its declaration accuracy is higher here than at home. Two things are not evidence and are not claimed as such: the mirror cell cannot see a defect that hits both copies equally, and no decision-level probe of KRAKEN's hosting was built. Two asymmetries remain. A cardless KRAKEN seat is not asked to declare on its own turn, because KRAKEN's on-turn loop does not implement cardless declaration, though it is still polled off-turn; the `--no-cardless-declare` cell removes that rule for both sides at once and moves the result by +0.127 in KRAKEN's favour on different deals, within the battery's unpaired resolution. And in FishLab's forced endgame, when KRAKEN's own decision names a different half-suit from the one the ladder's last rung demands, the engine's default all-to-one-seat guess is applied in KRAKEN's name and counted; that counter is not printed by the match driver, so it is bounded here instead: KRAKEN makes 0.406 forced declarations a game under its own dialect at 91.8% accuracy (0.029 at 71.8% under FishLab's), so even if every wrong one were such a substitution the effect is at most 0.067 sets per game (0.016 under FishLab's rules), which does not reach the result. If KV's project finds a defect in `kv6.hpp` the way this report found one in theirs, the paired instrument to price it is the same one used here.

6.4 What a cross-engine measurement should require of its bridge

The general lesson is the one the paper itself states and then did not apply: a number measured through a bridge is a claim about the bridge. Four checks would have caught this defect, and any of them is cheap.

1. **Reconstruct against the native agent, not against a copy of the reconstruction.** At the first decision of a game, a fresh agent and the live agent have identical state; they must answer identically, and the comparison must be made on positions from real games, after transfers have happened.
2. **Round-trip every reconstructed input.** A hand handed to `reset` must, replayed forward through the history, reproduce the hand the arbiter holds. This is one line and fails loudly.

3. **Host in both directions and require agreement.** Two independently written engines that disagree by three sets a game on the same pairing are reporting a bridge defect, not a dialect.
4. **Account for errors by class.** A foreign bot that suddenly declares half-suits its opponents hold is not exhibiting a subtle inference weakness; it is being lied to. Ownership versus allocation errors, and forced versus voluntary declarations, separate the two in one table.

Legality checks and fallback counters, the checks the paper relied on, are necessary and not sufficient: a scrambled belief plays legal moves.

7 Conclusion

KRAKEN v1.1's cross-engine headline is an artifact of its bridge into SESTINA v1.0, which reset the opponent with the hand it held rather than the hand it was dealt and thereby corrupted every certificate the opponent's belief is built from. The paper's number reproduces on its own apparatus; correcting one line and replaying the same deals moves it by -3.149 sets per game and reverses its sign; FishLab's arbiter, with KRAKEN hosted exactly as its guide demands, gives the same answer independently; and the defect is visible decision by decision. Under each of the dialects tested, SESTINA v1.0 beats KRAKEN by roughly two thirds of a set per game and wins about three games in five, and KRAKEN sits below every FishBot release since v0.4. A patch for their bridge accompanies this report.

A Reproduction and artifacts

FishLab is at `https://github.com/dylann4500/fishbot`; this report was produced at commit `f9b4743` with the uncommitted additions in the working tree at the time, none of which touch the measured engine core. KV's repository is at `https://github.com/kv1514/fish-researchp12` (branch `claude/fishnbot-work-access-g7ciey`, head `5da2040`). All artifacts, drivers and this document's number generator are under `research/kraken-check/`:

`e1_results.jsonl` one row per FishLab block, with the exact command line; `run_e1.py` regenerates it.

`e2_*.jsonl` one row per game in KV's arbiter, per bridge variant, with per-side ask, declaration, forced-declaration and error-class counts; `kvarena/run_e2.py` and `kvarena/e2_variants.py` regenerate them after `stage_kvarena.sh` has copied KV's package into a work directory and compiled their shim (and the ladder-capable `shim_decide2.cpp`) against `engine/src`.

`e5_probe.json`, `agree_probe.cpp` the decision probe.

`e4_upstream_777001.json` upstream KRAKEN v1.1 on the vendored block's seed.

`bridge_kraken.jsonl` a full request–reply transcript of `kv6.hpp` for one deal.

`upstream/` the paper's headline result and 10,000-game journal, the bridge ledger, and the bridge file the patch applies to.

`dylan_v07_dealt_hand.patch` the repair (bridge revision 3).

`analyse.py`, `make_numbers.py`, `make_report.py` the aggregation, the macros and tables of this document, and the Markdown report `docs/KRAKEN-CLAIM-CHECK.md`.

The commands, from the repository root:

```
python3 research/kraken-check/run_e1.py 9   # argument = threads (no effect on results)
sh research/kraken-check/stage_kvarena.sh && cd engine/external/kvarena-work
PY=../kv-venv/bin/python
OUT=../../../research/kraken-check
E2_VARIANT=rev2      $PY run_e2.py 1000 6 910000 $OUT/e2_rev2.jsonl
E2_VARIANT=dealthand $PY run_e2.py 1000 6 910000 $OUT/e2_dealthand.jsonl
for v in ladder seedvary dealtseed; do
  E2_VARIANT=$v $PY run_e2.py 1000 6 910000 $OUT/e2_$v.jsonl; done
E2_VARIANT=dealthand $PY run_e2.py 4000 10 920000 $OUT/e2_dealthand_big.jsonl
sh $OUT/run_probe.sh                        # the decision probe (e5)
cd ../../..
python3 research/kraken-check/analyse.py
python3 research/kraken-check/make_numbers.py
```

B The repair

The whole of the patch to `fish4/dylan_v07.py` is the following method and its use in `_feed` in place of `obs.hand`:

```
@staticmethod
def _dealt_hand(obs):
    me, hand = obs.player, obs.hand
    for ev in reversed(obs.history):
        if isinstance(ev, AskEvent) and ev.success:
            if ev.asker == me:      hand &= ~(1 << ev.card)
            elif ev.target == me:   hand |=  (1 << ev.card)
        elif isinstance(ev, ClaimEvent):
            base = ev.half_suit * CARDS_PER_HALF_SUIT
            for i, holder in enumerate(ev.revealed):
                if holder == me:    hand |=  (1 << (base + i))
    return hand
```

KV's own `Observation` class already provides exactly this walk as `initial_hand()`, used by their information-leakage test, so the repair is also the one-token change `obs.hand` to `obs.initial_hand()` in `_feed`; the patch spells the walk out so that it does not depend on that helper. It uses only public events and the seat's own hand, so it is available to any host; a host whose engine does not reveal holders on a failed declaration can substitute the declared holders for a successful one and the seat's own knowledge of which of its cards were retired for a failed one.

References

[1] Kausthubh Veldanda. *Exact Inference Is Not Enough: Opponent Models and Endgames in Six-Player Literature*. KRAKEN v1.1, 30 August 2026. `paper/kraken.pdf` in the repository https://github.com/kv1514/fish-researchp12.

[2] Dylan Nguyen. *SESTINA v1.0: Exact Inference, Robust Evaluation, and Adversarial Testing in Six-Player Canadian Fish*. FishLab technical report, 31 August 2026. `output/pdf/sestina_v10.pdf` in the repository https://github.com/dylann4500/fishbot.